USER MANUAL

B-FUN-500 GLaDOS

Introduction

- **Overview of the Project**: Shift from LISP to LUA interpreter.
- **Purpose and Capabilities**: Emphasize the LUA interpreter's features.
- **Scope**: Focus on LUA language interpretation and compiler construction in Haskell.

Getting Started

- System requirements.
- Detailed installation steps.
- Initial configuration and setup.

Basic Usage

- Starting the Interpreter.
- Writing LUA Scripts.
- Executing Programs.

Advanced Features

- Abstract Syntax Tree (AST)
- Custom functions and extensions.
- Evaluating Expressions.
-

Advanced Features

- Detailed explanation of advanced features.
- Custom functions and extensions.
- Optimization techniques.
- Parser Mechanics.

Troubleshooting and FAQs

- Common issues and their solutions.
- Frequently asked questions.

Examples and Scenarios

- Basic LUA Operations.
- Defining and Using Functions.

- Complex Expressions
- Parser Functionality

Contributing to the Project

- Guidelines for contributing.
- How to submit bugs or feature requests.

Appendix and Additional Resources

- References to external resources.
- Glossary of terms.

Introduction

- **Overview**: The GLaDOS project is a Haskell-based programming assignment designed to develop a language interpreter. Initially focused on creating a minimalist LISP interpreter, the project has pivoted to the LUA language, expanding its capabilities and providing an enriched learning experience in language interpretation and compiler construction.

- **Objectives**: The primary objective is to familiarize users with the principles of language interpretation and compiler construction, using Haskell as the implementation language. This shift to LUA adds a layer of complexity and practicality, enriching the educational experience.

- **Scope**: The B-FUN-500 GLaDOS project now centers on building a minimalist LUA interpreter within the Haskell environment. This direction allows users to delve into the specifics of LUA language interpretation, extending their understanding of compiler

theory and Haskell programming. The project involves interpreting LUA scripts, thereby offering a hands-on approach to learning the intricacies of compiler construction and language interpretation.

Getting Started

System Requirements

- **Operating System**: Compatible with major operating systems (Windows, macOS, Linux).
- **Haskell**: Haskell Tool Stack installed, preferably the latest version.
- **Memory and Processor**: Adequate to run Haskell programs, with no specific high-end requirements.

Installation Steps

1 - Clone Repository: Use git clone **git@github.com:EpitechPromo2026/B-FUN-500-PAR-5-2-glados-nicolas.poupon.git glados** to clone the project to your local machine.

2 - Navigate to Directory: Change directory to the project folder using **cd glados.**

3 - Install Dependencies: Run stack setup to install the necessary Haskell dependencies.

Initial Configuration

- **Building the Project**: Use **stack build** to compile the project.
- **Running Tests**: Ensure functionality by running unit tests with **stack test --coverage**.
- **Starting the Interpreter**: Use **stack exec glados** to start the LUA interpreter.

Basic Usage

Starting the Interpreter: Launch the interpreter by running stack exec glados in the command line within the project directory.

Writing LISP Expressions:

- LISP syntax basics: Understand the fundamental syntax of LUA, like parentheses usage, symbols, and list structures.
- Simple commands: Begin with basic LUA expressions such as arithmetic operations (e.g., (1 + 2) for addition).

Executing Programs:

    - Run commands: Input LUA expressions directly into the interpreter to execute them.
    - Interpreting results: Learn how to read and understand the output provided by the interpreter.

Exiting the Interpreter: Use an exit command or close the terminal window to exit the interpreter.

Advanced Features

Abstract Syntax Tree (AST) and S-Expressions:

- Understand the AST structure defined in AST.hs, crucial for representing LUA expressions.
- Explore the conversion of S-Expressions to AST, a key step in the interpretation process.

Custom Function Definitions:

- Learn how to define custom functions within the interpreter using the Define structure in AST.hs.
- Explore the syntax and semantics of function definition and invocation in LUA.

Evaluating Expressions:

- Dive into the evalAST function in AST.hs for insights into how LUA expressions are evaluated.
- Explore arithmetic operations, symbol evaluation, and function calls.

Parser Mechanics:

- Examine the Parser.hs for understanding the parsing logic.
- Look into how LUA expressions are parsed and processed into AST.

Troubleshooting and FAQs

- **Common Issues**: List common issues or errors users might encounter and their solutions.
- **FAQs**: Include a section answering frequently asked questions related to usage, features, and troubleshooting.

Examples and Scenarios

Basic Arithmetic Operations:

- **Example**: Evaluating a simple addition 5 + 3.

- **Scenario**: Use the interpreter to perform basic arithmetic like addition, subtraction, multiplication, and division
- **Interaction**:
  - Start the interpreter **stack exec glados**.
  - To perform addition, enter: 5 + 3.
  - To perform subtraction, enter: 10 - 4.
  - For multiplication, enter: 6 * 7.
  - For division, enter: 20 / 5.
  - The interpreter should evaluate and display the results for each operation.

Defining and Using Custom Functions:

- **Example**: Creating a custom function for calculating factorial.
- Scenario: Show how to define a new function in LUA syntax and use it to calculate factorial of a number.
- **Interaction**: Define a factorial function using LUA's function definition syntax. For instance, define **factorial(n)** and then calculate **factorial(5)**

  Here is a code snippet for a factorial function:

  ```
  function factorial(n)
  if n == 0 then
  return 1
  else
  return n * factorial(n - 1)
  end
  end
  ```

  This code recursively calculates the factorial of a given number by multiplying it with the factorial of the preceding number, ultimately reaching the base case of 0 and returning 1.

Calculate the factorial of 5 by entering: factorial(5).

The interpreter should return the factorial of 5, which is 120.

Complex Expressions Using AST:

- **Example**: Evaluating a nested LUA expression.
- **Scenario**: Demonstrate the interpreter's capability to process complex nested expressions in LUA and elucidate the underlying AST mechanism.
- **Interaction**:
  - Enter a nested expression: (2 + 3) * (4 - 1).

    - The interpreter should correctly evaluate and display the result, which is 15.

Parser Functionality Test:

- **Example**: Parsing a complex LUA expression.
- **Scenario**: Illustrate how the interpreter parses LUA scripts, including variables, function calls, and control structures.
- Interaction:
    - Enter a complex LUA script, for example:

      ```
      local x = 10
      local y = 20
      function add(a, b)
        return a + b
      end
      print(add(x, y))
      ```

      The script declares variables, defines a function, and makes a function call.

      The interpreter should parse this script, execute it, and display the result of the function call, which in this case is 30.

Contributing to the Project

- **Contribution Guidelines**: Outline how users can contribute to the project, including coding standards and pull request procedures.
- **Reporting Bugs**: Instructions on how users can report bugs or suggest features.

Appendix and Additional Resources

- **Glossary**: Include a glossary of terms used in the manual and project.
- **External Resources**: Links to external resources for further reading on Haskell, LISP, and interpreter design.
-